

Recursividad y Árboles

Programación II

Recursividad

Recursividad

- Tiene uno o más **Casos Bases**
- Se llama a sí mismo reduciendo el problema hasta llegar a un **Caso Base**. Esto se conoce como **Llamada Recursiva**.
- Todo algoritmo **Recursivo** se puede escribir de forma **iterativa** y viceversa.
- Los problemas de naturaleza recursiva tienden a ser más complicados de plasmar en algoritmos iterativos.

Recursividad

Algunos problemas de naturaleza recursiva:

- Factorial
- Fibonacci
- Búsqueda Binaria
- Insertar en una lista enlazada ordenada
- etc...

Recursividad - Factorial

```
int factorial(int n) {  
    if (n==0) // caso base  
        return 1;  
    return n * factorial(n-1);  
}
```

```
public int factorial(int n) {  
    int ret = n;  
    for (int i=n-1;i>1;i--)  
        ret = ret * i;  
    return ret;  
}
```

Recursividad - Fibonacci

```
int fib(int n) {  
    if (n < 2) // f0= 0 y f1= 1  
        return n;  
    return fib(n-1) + fib(n-2);  
}
```

$$O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$$

```
public int fib(int n) {  
    if (n<2)  
        return n;  
    int n0=0;  
    int n1=1;  
    int fibn=n1+n0;  
    for (int i=2; i<n; i++) {  
        fibn = n1 + n0;  
        n0=n1;  
        n1=fibn;  
    }  
    return fibn;  
}
```

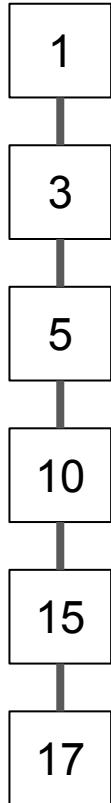
$$O(n)$$

Recursividad - Búsqueda Binaria

```
int busqBin(int[] A, int x) {  
    return busqBin(A,x,0,A.length-1);  
}  
  
int busqBin(int[] A, int x,  
            int ini, int fin) {  
    if (ini>fin)  
        return -1;  
    int medio = (ini+fin) / 2;  
    if (A[medio] < x)  
        return busqBin(A,x,medio+1,fin);  
    if (A[medio] > x)  
        return busqBin(A,x,ini,medio-1);  
    return medio;  
}
```

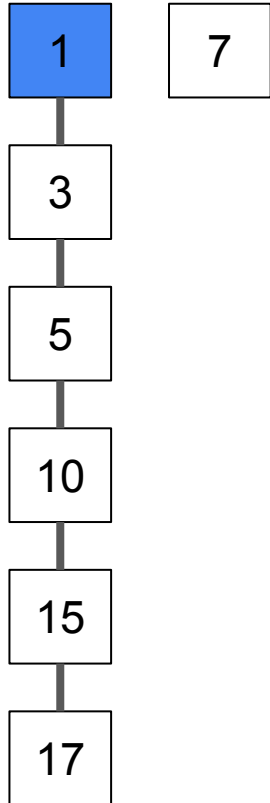
```
int busqBin(int[] A, int x) {  
    int ini=0;  
    int fin=A.length-1;  
    int pos=-1;  
    int medio;  
  
    while (ini < fin && pos == -1) {  
        medio = (ini+fin) / 2;  
        if (A[medio] < x)  
            ini = medio+1;  
        else if (A[medio] > x)  
            fin = medio-1;  
        else  
            pos=medio;  
    }  
    return pos;  
}
```

Recursividad - Insertar en lista ordenada



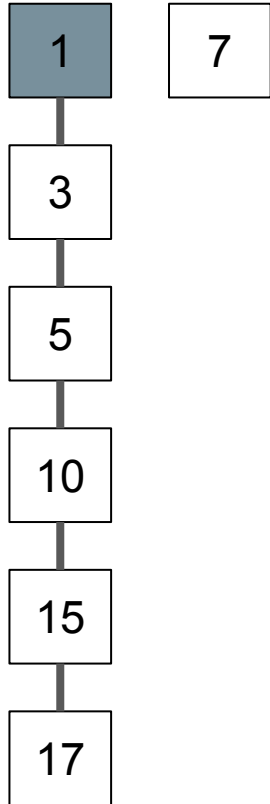
```
void agregar(int valor) {  
    raiz = agregar(raiz,valor);  
}  
  
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next,valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



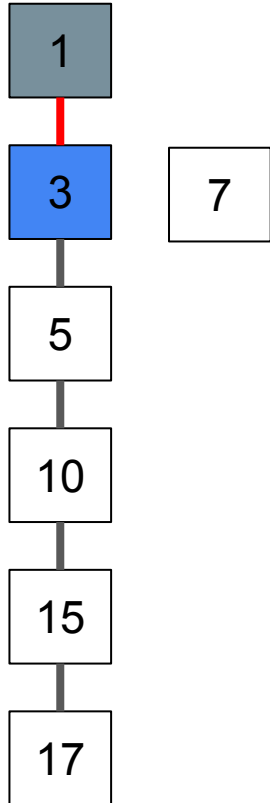
```
void agregar(int valor) {  
    raiz = agregar(raiz,valor);  
}  
  
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next,valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



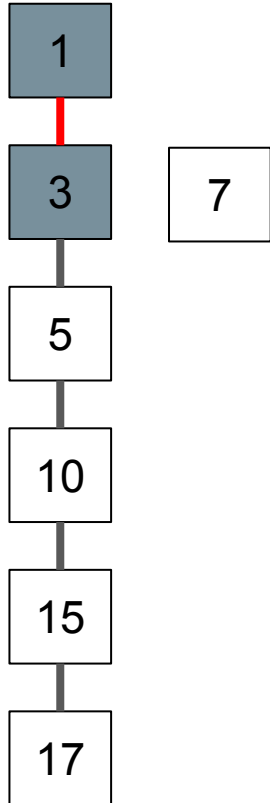
```
void agregar(int valor) {  
    raiz = agregar(raiz,valor);  
}  
  
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next,valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



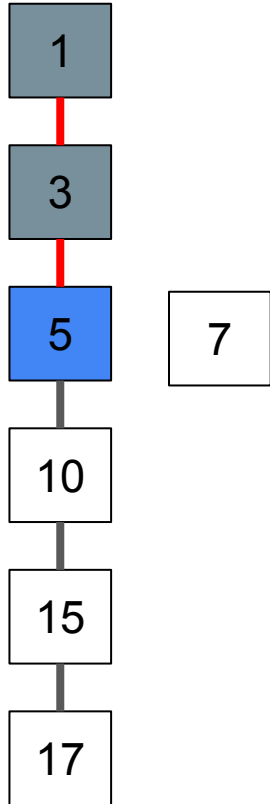
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



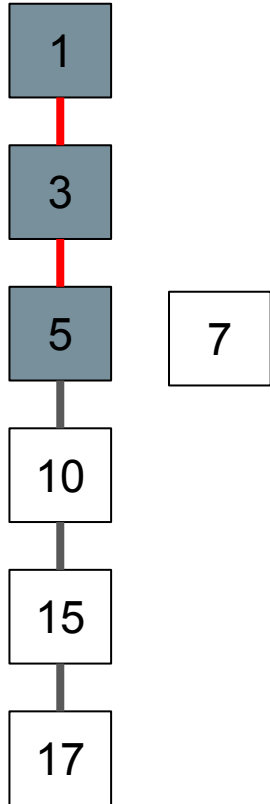
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



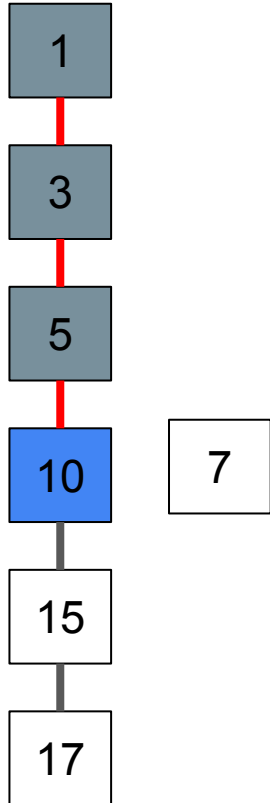
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



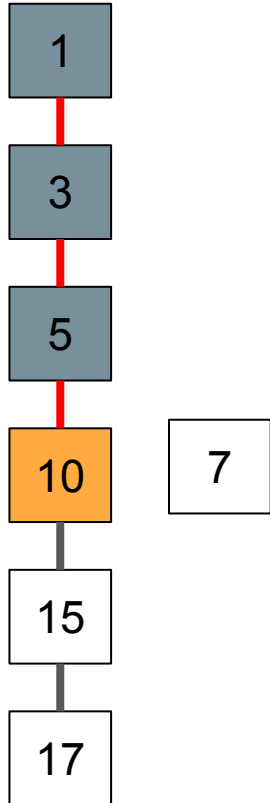
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



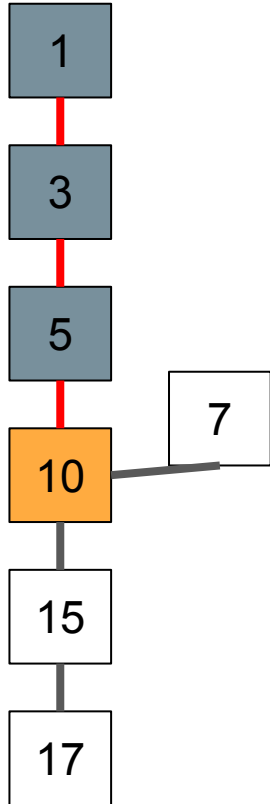
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



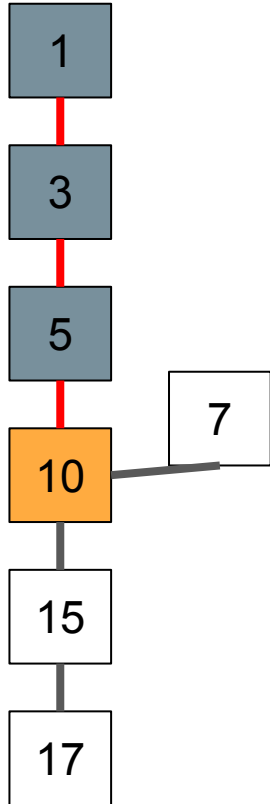
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



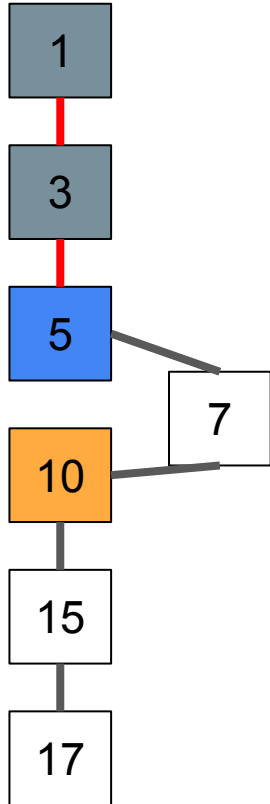
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



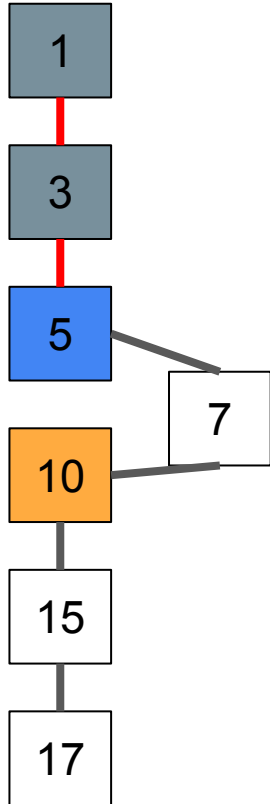
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



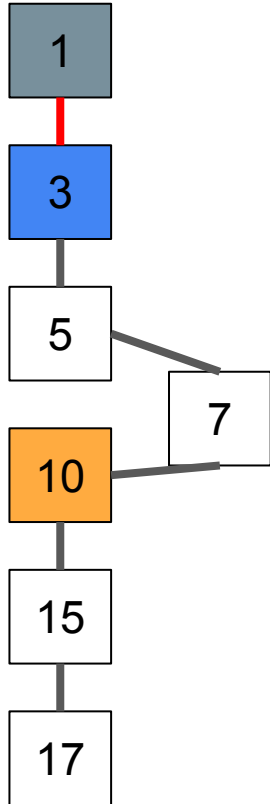
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



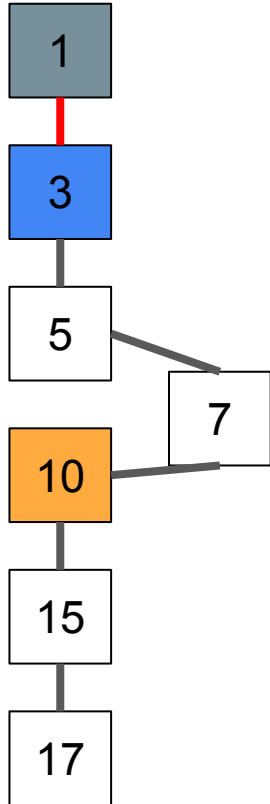
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



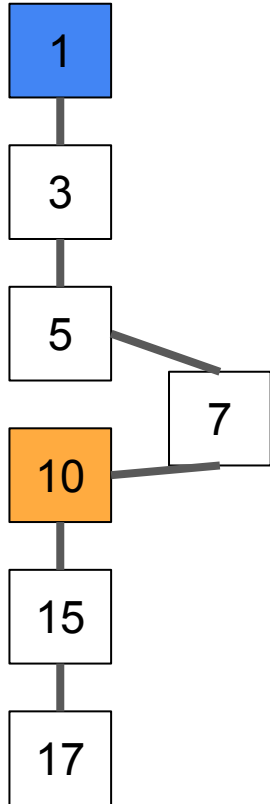
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



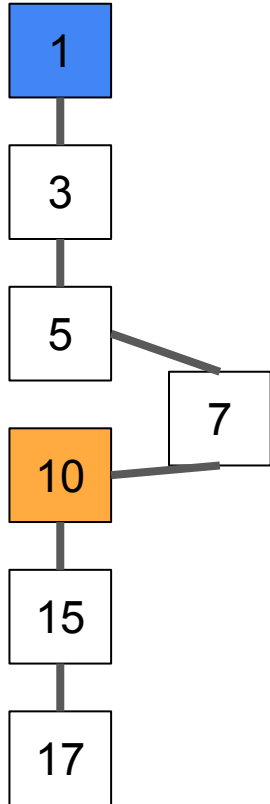
```
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



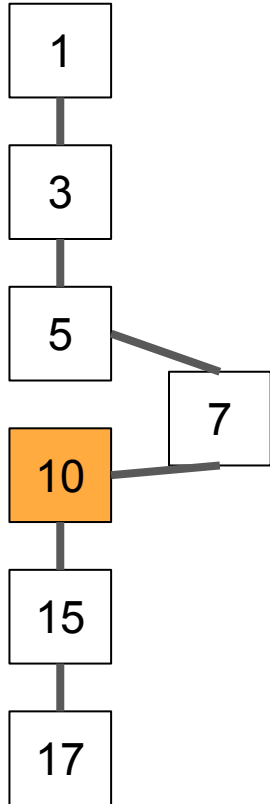
```
void agregar(int valor) {  
    raiz = agregar(raiz,valor);  
}  
  
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo==null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next,valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next=nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



```
void agregar(int valor) {  
    raiz = agregar(raiz, valor);  
}  
  
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo == null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next = nodo;  
        return nuevoNodo;  
    }  
}
```

Recursividad - Insertar en lista ordenada



```
void agregar(int valor) {  
    raiz = agregar(raiz, valor);  
}  
  
private Nodo agregar(Nodo nodo, int valor) {  
    if (nodo == null)  
        return new Nodo(valor);  
  
    if (nodo.elem < valor) {  
        nodo.next = agregar(nodo.next, valor);  
        return nodo;  
    } else {  
        Nodo nuevoNodo = new Nodo(valor);  
        nuevoNodo.next = nodo;  
        return nuevoNodo;  
    }  
}
```

¿Consultas?

Árboles

Árboles

- Es una estructura de datos con forma de árbol.
- Todos los datos se almacenan en nodos.
- Cada nodo también apunta a uno o más **Nodos Hijos**.
- Parte de un **Nodo Raíz**.
- Todos los **Nodos** son accesibles desde el **Nodo Raíz**.
- No tiene ciclos.

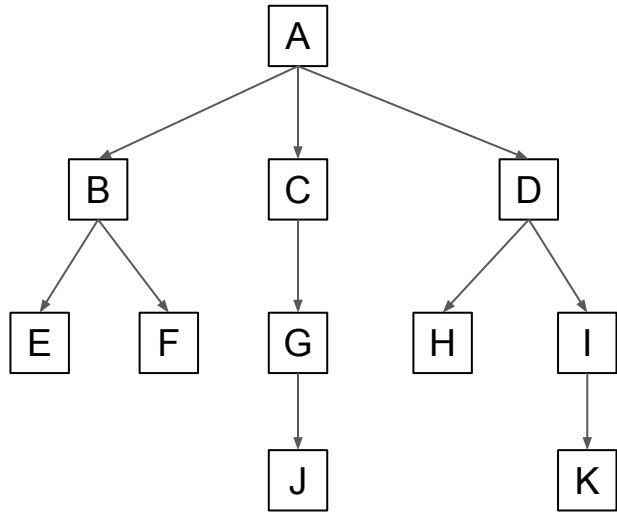
Árboles

Un árbol consta de un conjunto finito de elementos, denominados nodos, y un conjunto finito de líneas dirigidas, denominadas enlaces, que conectan los nodos.

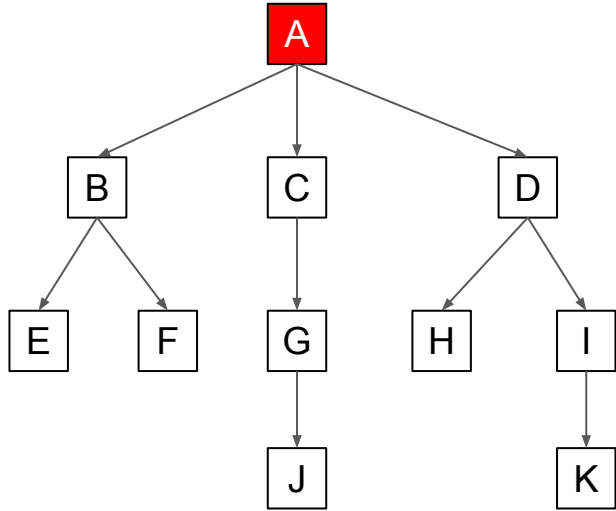
Hay 3 tipos de nodos:

- **Nodo Raíz:** El único sin padre.
- **Nodos Internos:** Nodos con uno o más hijos.
- **Nodos Hojas:** Nodos sin hijos.

Árboles

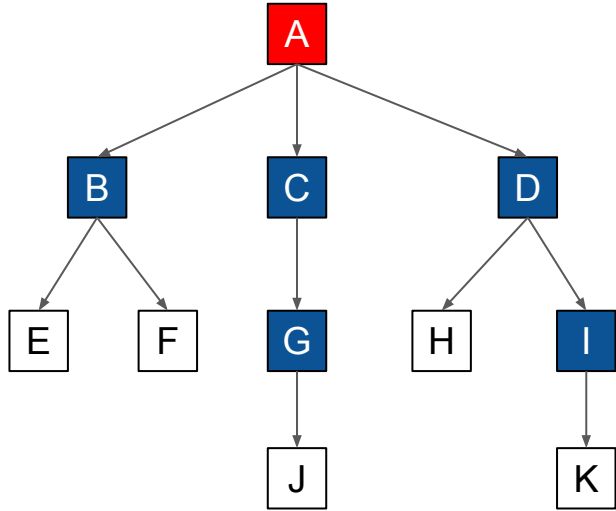


Árboles - Nodo Raíz



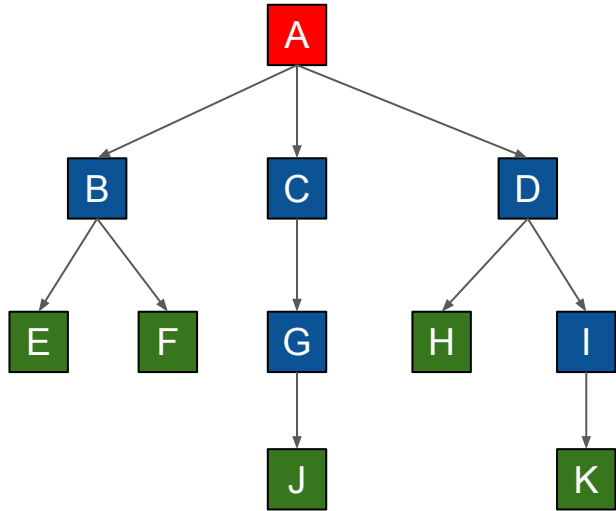
- **Nodo Raíz**

Árboles - Nodos Internos



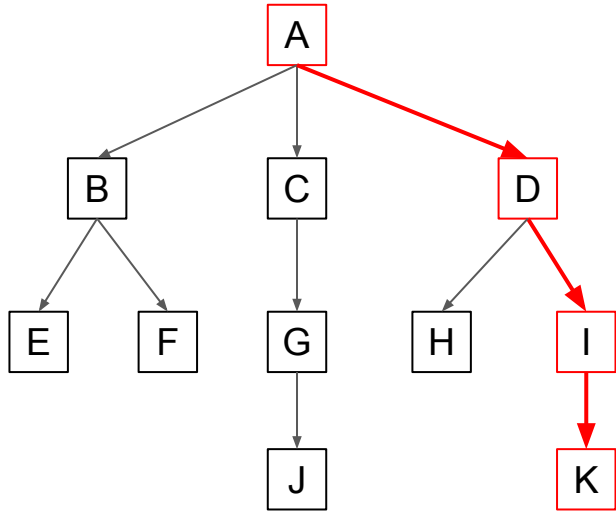
- **Nodo Raíz**
- **Nodos Internos**

Árboles - Nodos Hoja



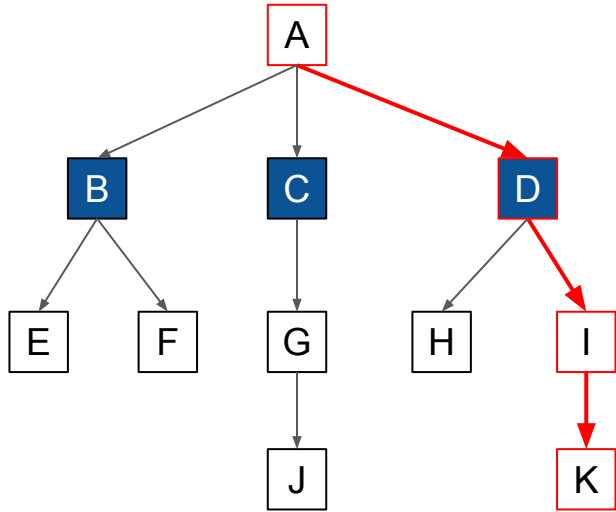
- **Nodo Raíz**
- **Nodos Internos**
- **Nodos Hojas**

Árboles - Altura



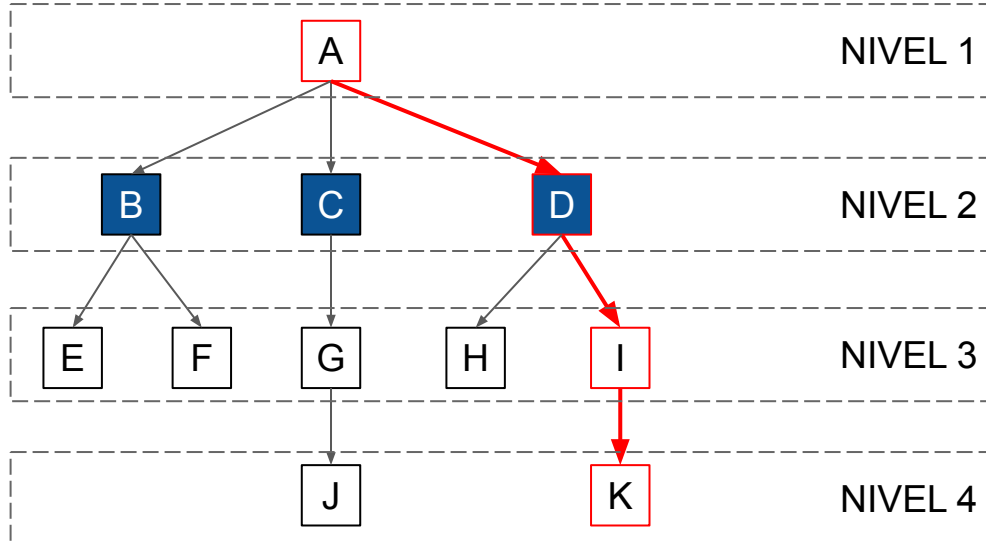
- Tiene altura 4

Árboles - Grado



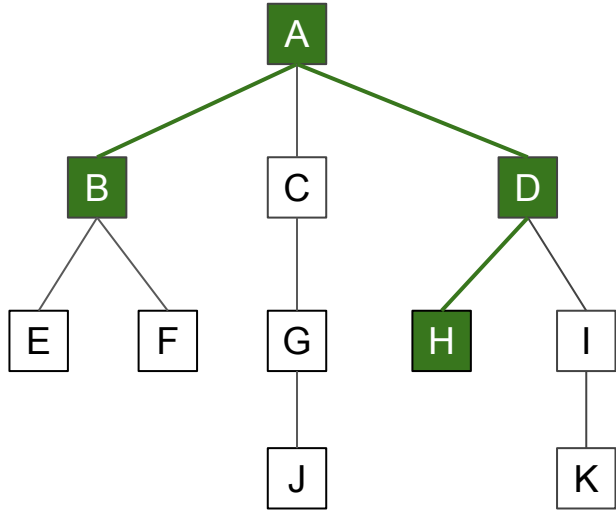
- Tiene **altura 4**
- Es de **Grado 3**

Árboles - Niveles



- Tiene **altura 4**
- Es de **Grado 3**

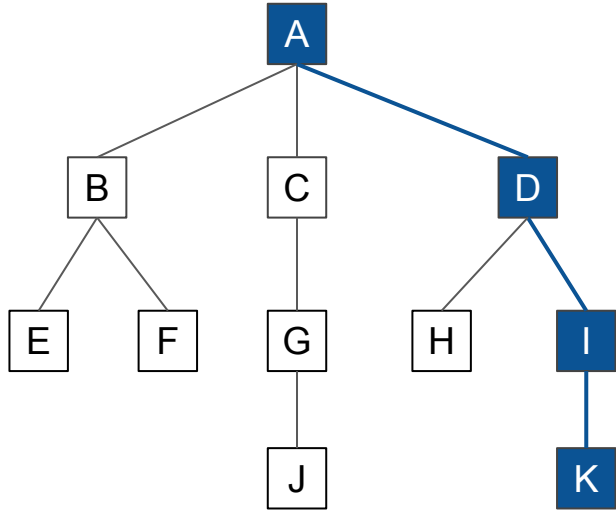
Árboles - Camino



Secuencia de nodos
adyacentes para llegar
de un nodo X a un nodo
Y.

Camino de B a H:
 $\{B, A, D, H\}$

Árboles - Rama



Camino que comienza en la raíz y termina en una Hoja.

Rama hasta la hoja K:
 $\{A,D,I,K\}$

Árboles - conceptos

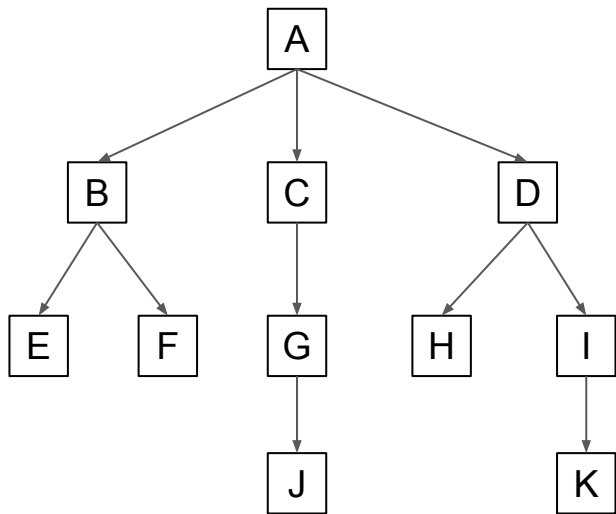
- Nodo **Raíz** - Nodo **Interno** - Nodo **Hoja**
- **Grado**: cantidad máxima de hijos que puede tener un nodo.
- **Altura**: cantidad de nodos desde la raíz hasta la hoja más lejana. Longitud de la rama más larga.
- **Rama**: camino desde la raíz hasta una hoja
- **Camino**: secuencia de nodos adyacentes que conectan 2 nodos(X, Y).
- **Nivel**: todos los nodos que están a la misma distancia de la **Raíz**.

AB - Árbol Binario

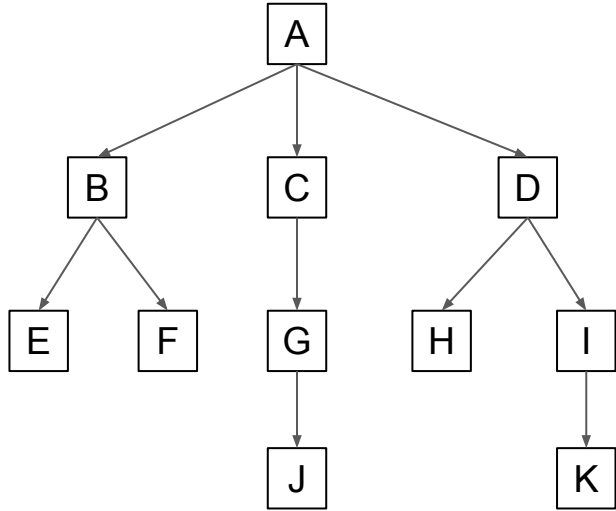
AB - Árbol Binario

- Es un Árbol de **Grado 2**.
- Y además:
 - Es un Árbol vacío.
 - O, El subárbol derecho y el subárbol izquierdo son árboles binarios.
- OBS: Es una definición recursiva.

AB - ¿Es AB?

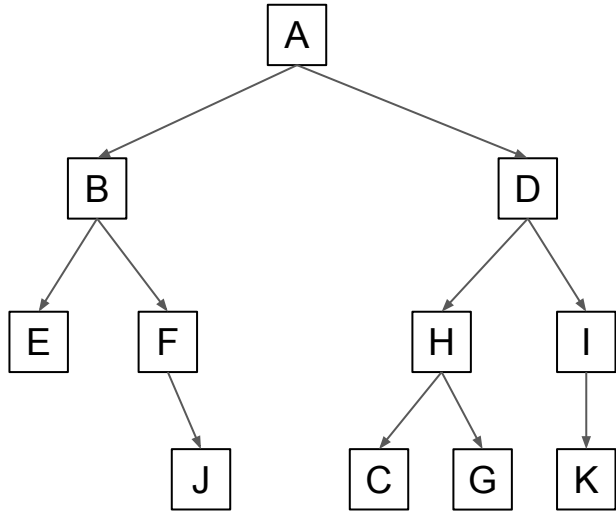


AB - ¿Es AB?



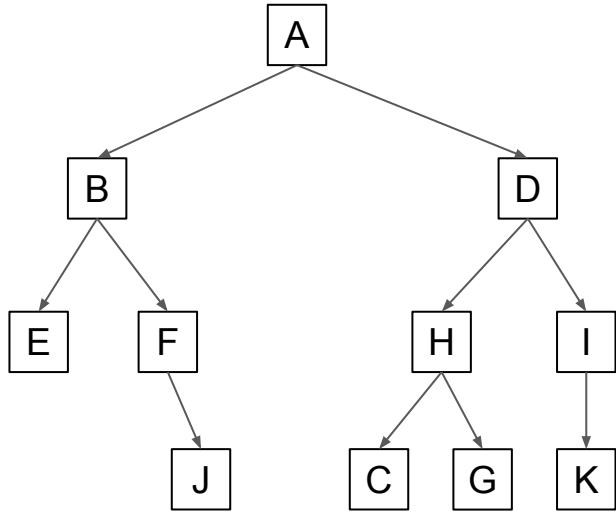
No es AB porque
es de Grado 3.

AB - ¿Es AB?



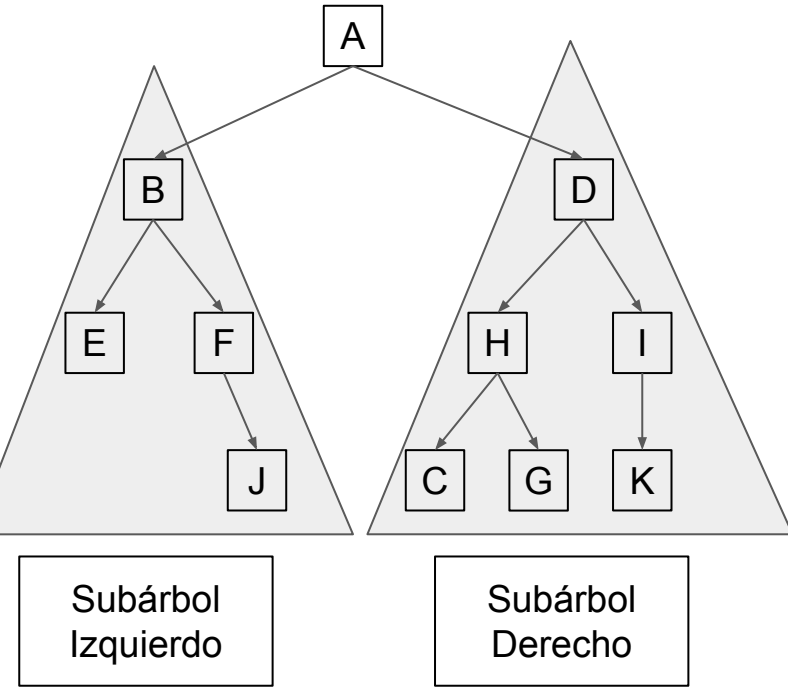
- Es de Grado 2?

AB - ¿Es AB?



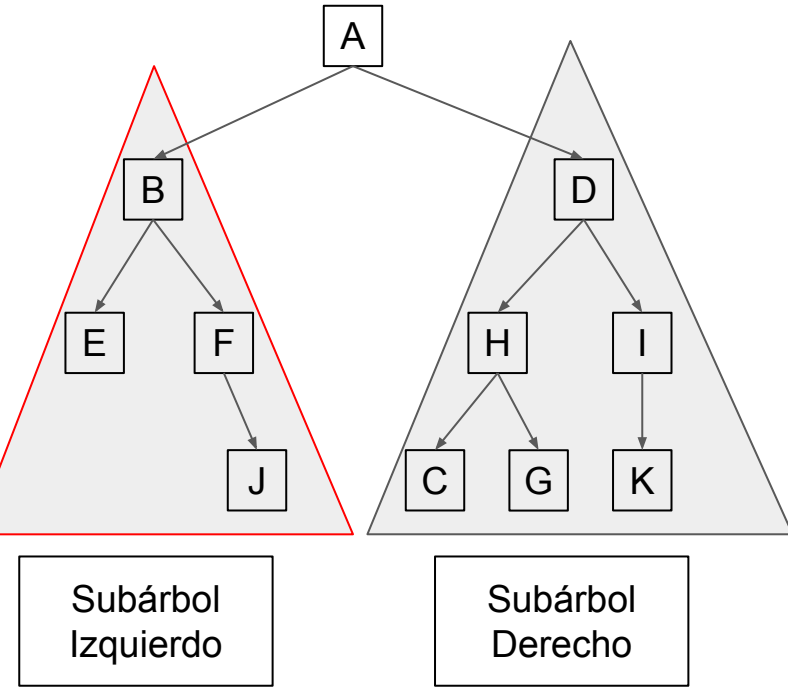
- Es de Grado 2.
- Sus subárboles son AB o son vacíos?

AB - ¿Es AB?



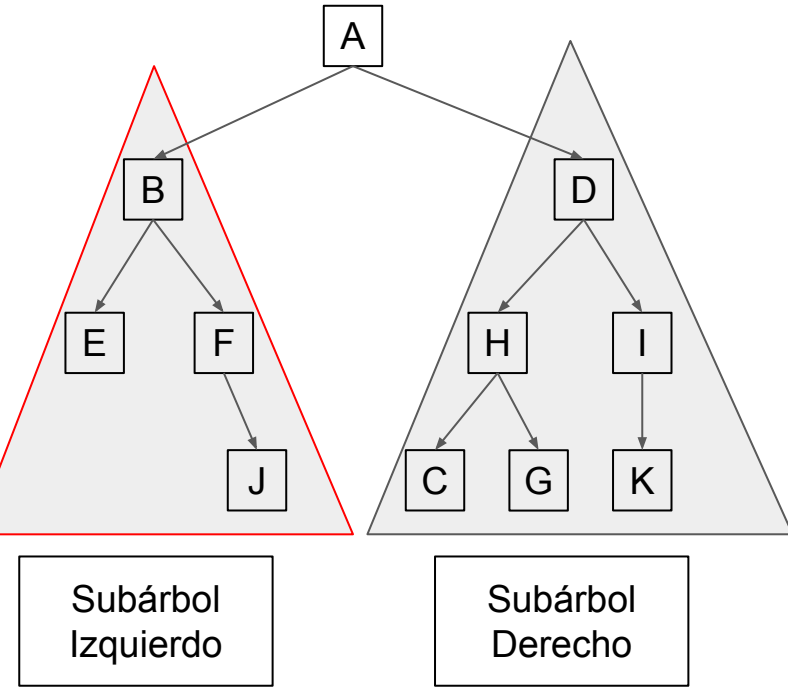
- Es de Grado 2.
- Sus subárboles son AB o son vacíos?

AB - ¿Es AB?



- Es de Grado 2.
- Sus subárboles son AB o son vacíos?

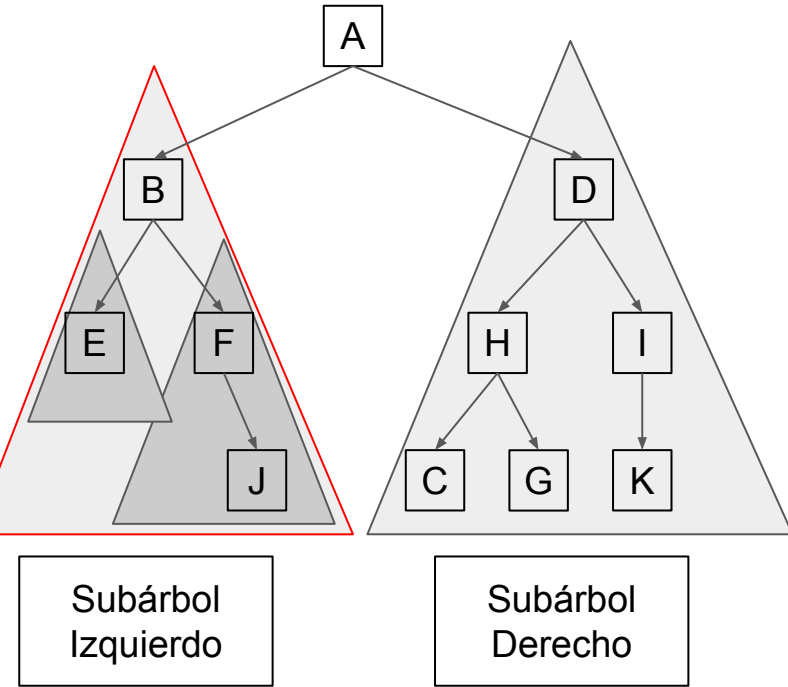
AB - ¿Es AB?



Subárbol B:

- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

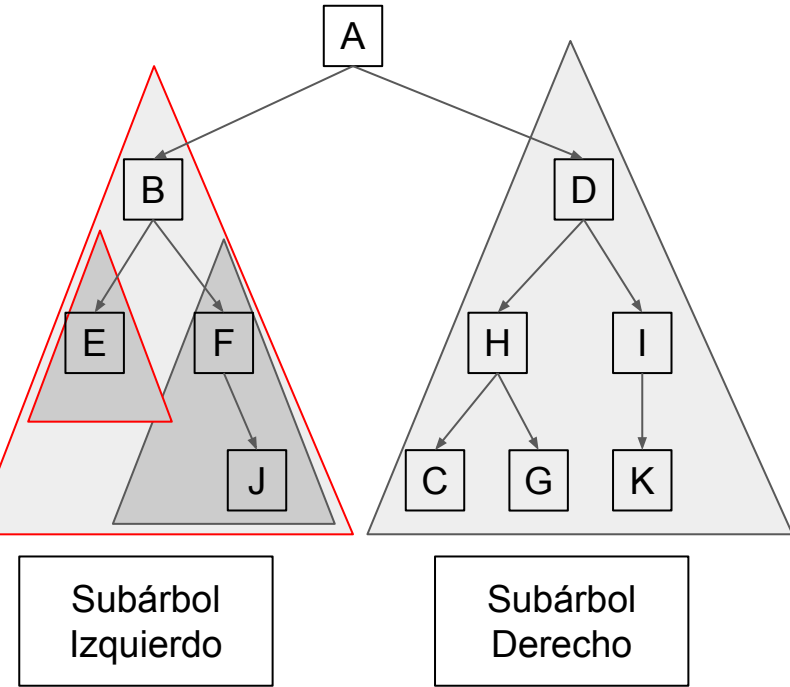
AB - ¿Es AB?



Subárbol B:

- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

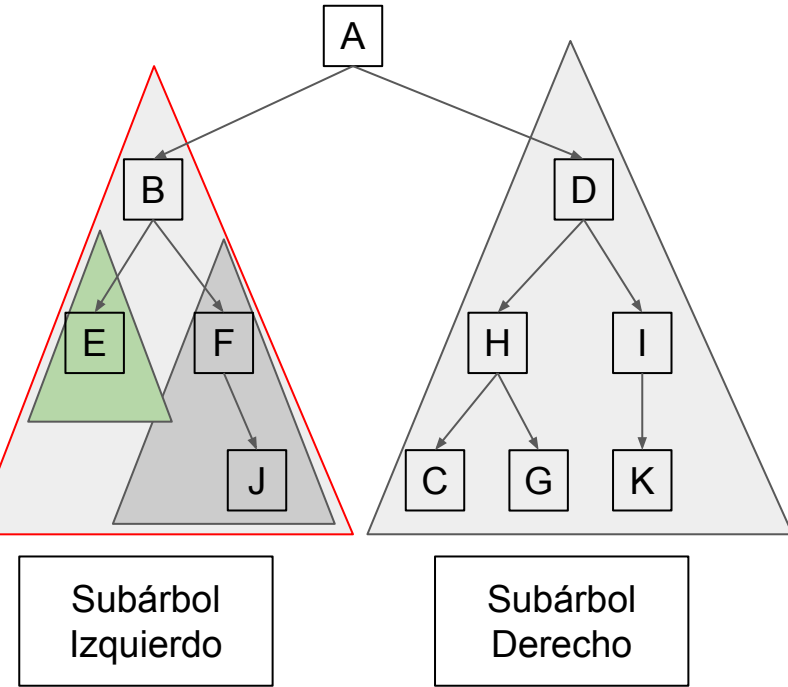
AB - ¿Es AB?



Subárbol E es AB?

- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

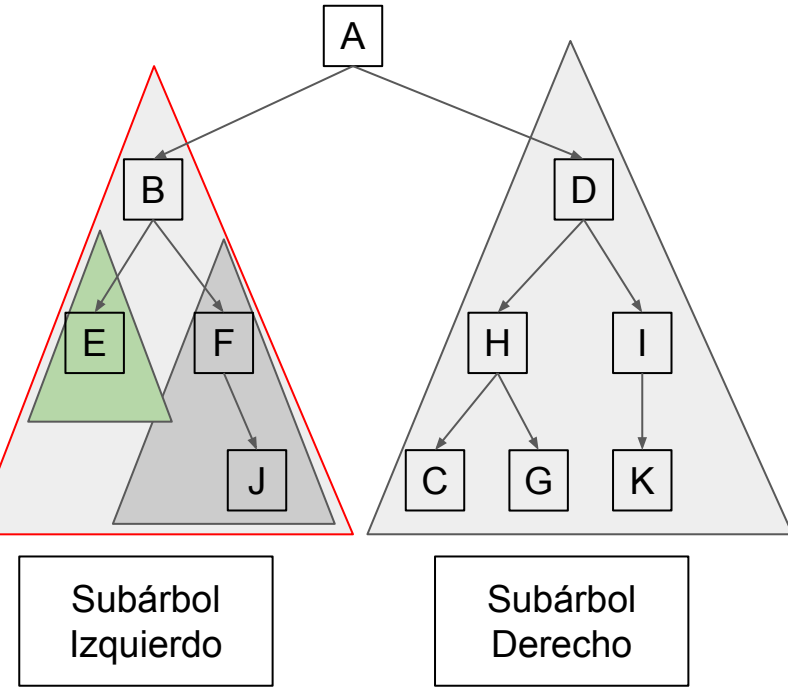
AB - ¿Es AB?



Subárbol E es AB?

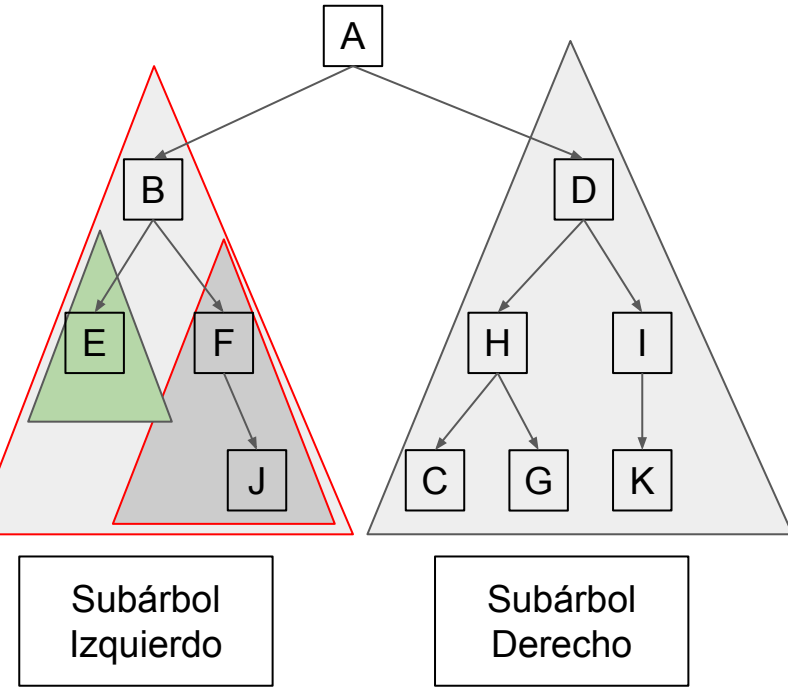
- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

AB - ¿Es AB?



- Subárbol izquierdo:
- Es de Grado 2?
 - Sus subárboles son AB o son vacíos?

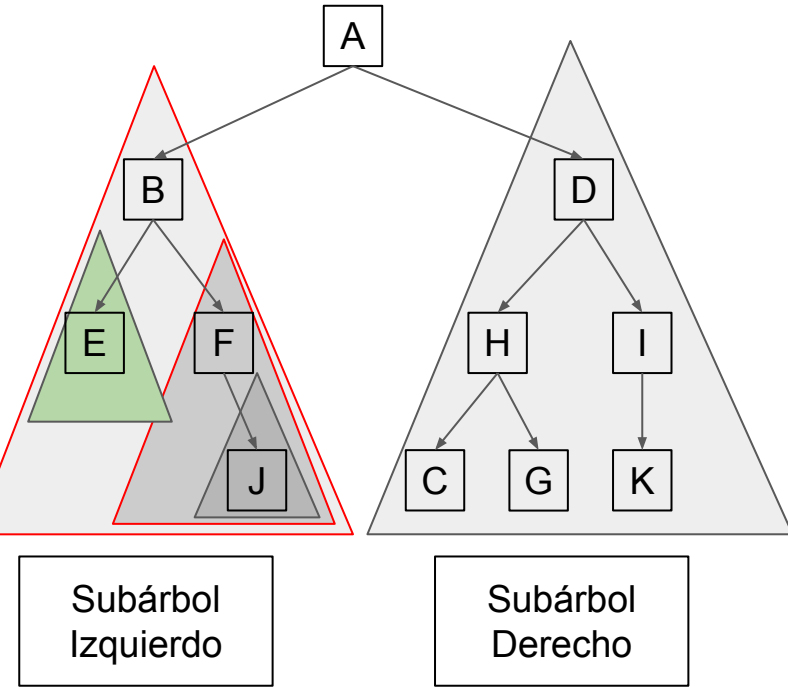
AB - ¿Es AB?



Subárbol F es AB?

- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

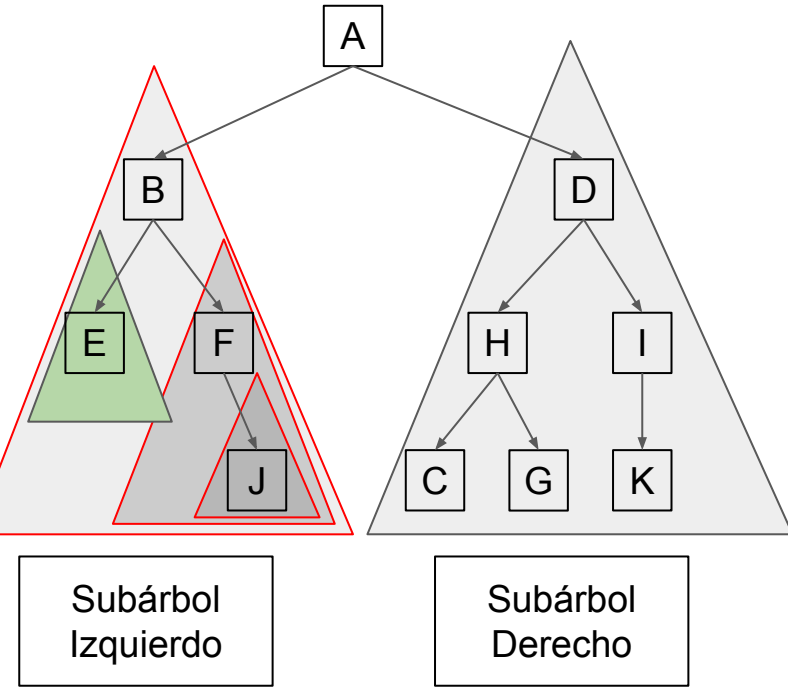
AB - ¿Es AB?



Subárbol F es AB?

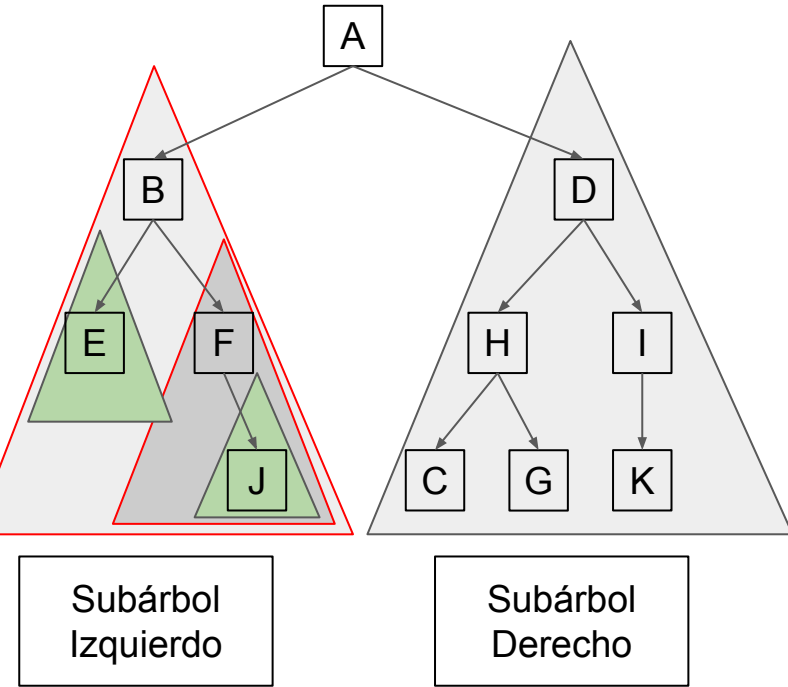
- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

AB - ¿Es AB?



- Subárbol J es AB?
- Es de Grado 2?
 - Sus subárboles son AB o son vacíos?

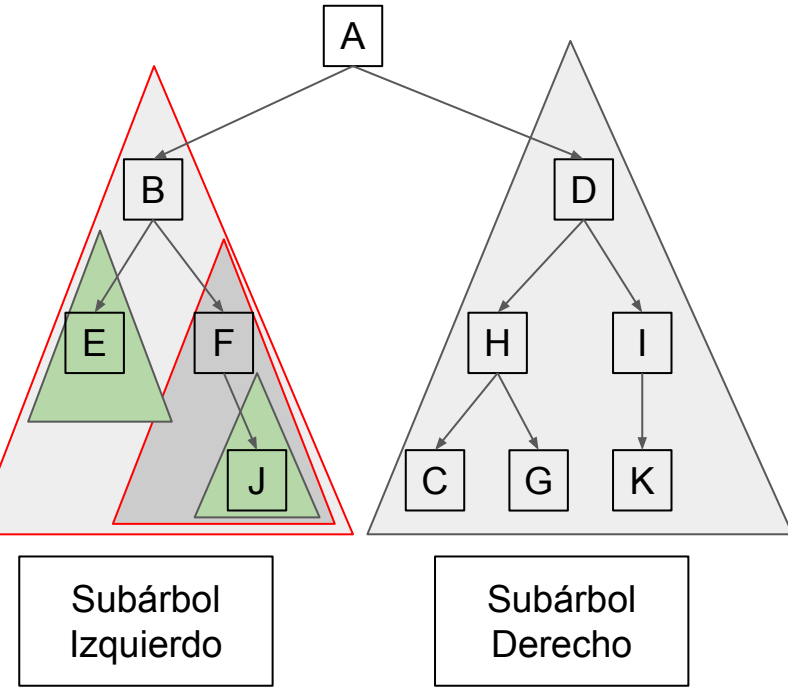
AB - ¿Es AB?



Subárbol J es AB?

- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

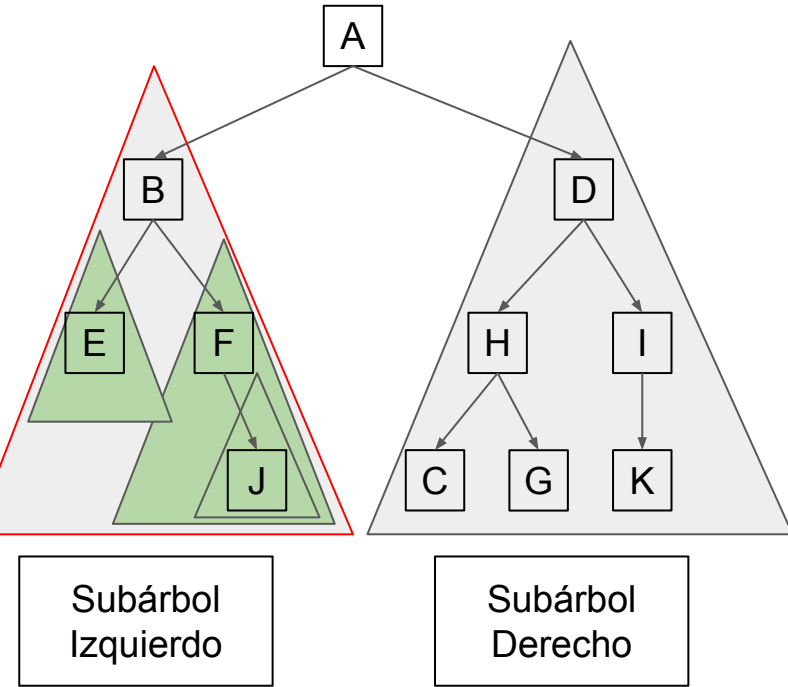
AB - ¿Es AB?



Subárbol F es AB?

- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

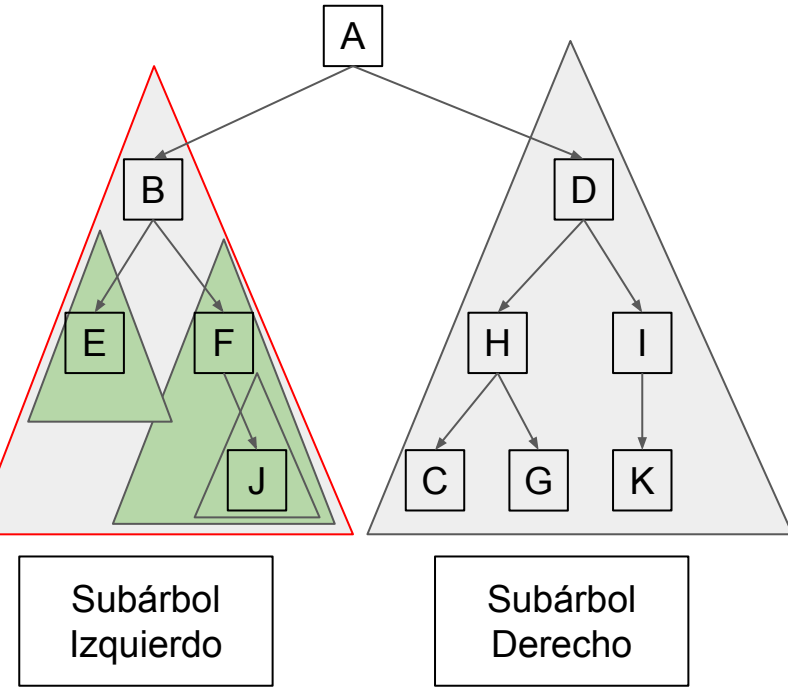
AB - ¿Es AB?



Subárbol F es AB?

- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

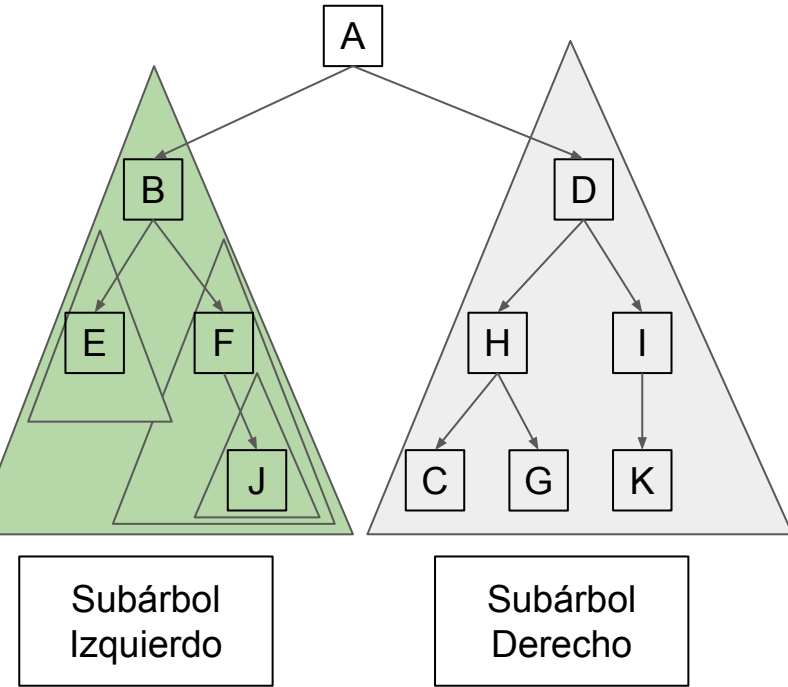
AB - ¿Es AB?



Subárbol B:

- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

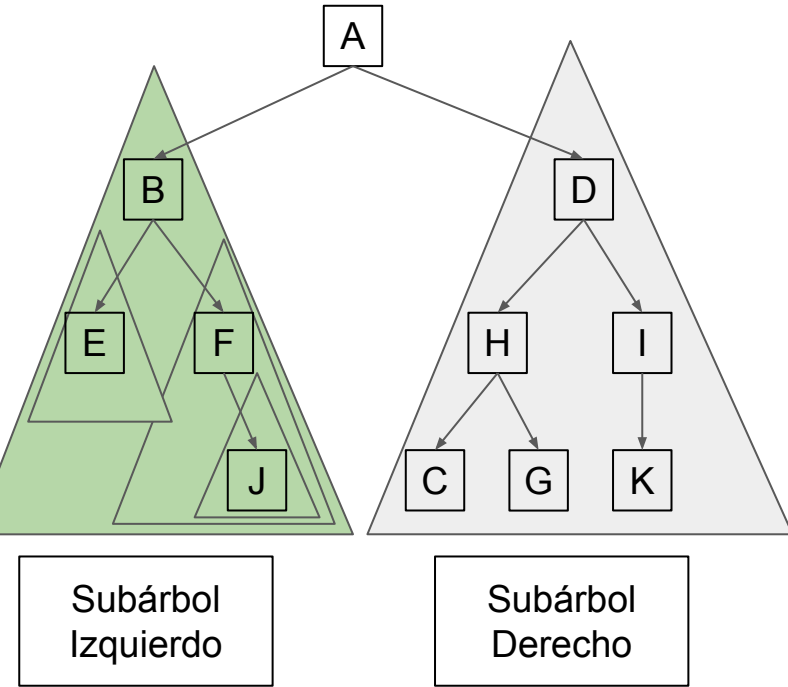
AB - ¿Es AB?



Subárbol B:

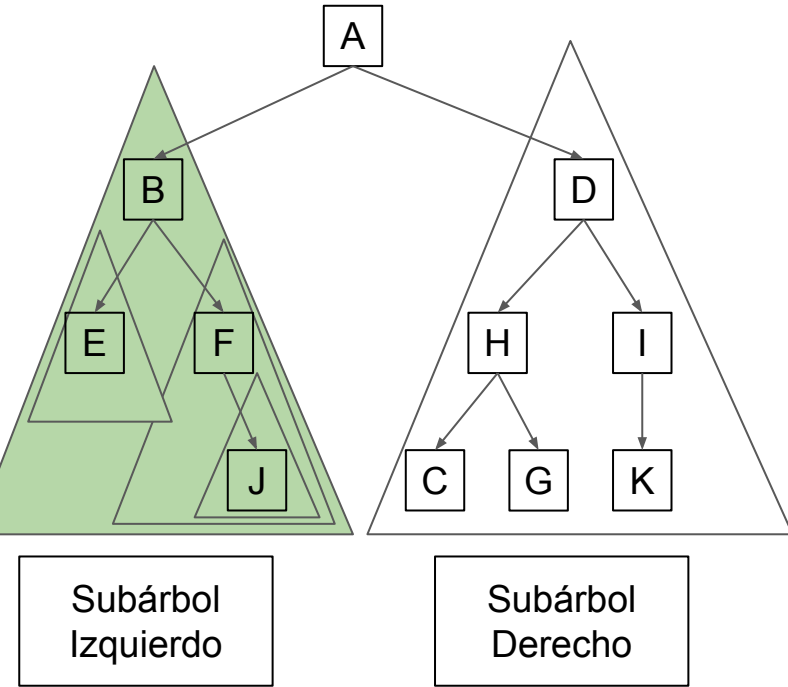
- Es de Grado 2?
- Sus subárboles son AB o son vacíos?

AB - ¿Es AB?



- Es de Grado 2.
- Sus subárboles son AB o son vacíos?

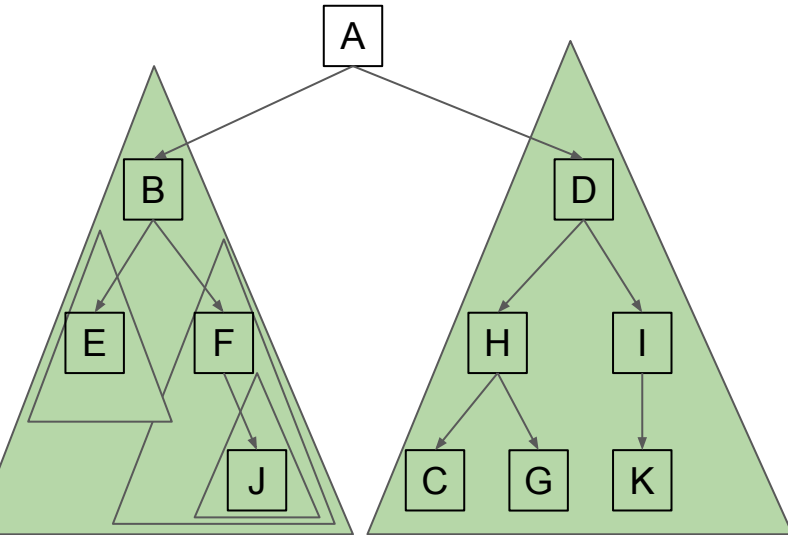
AB - ¿Es AB?



- Es de Grado 2.
- Sus subárboles son AB o son vacíos?

Si repetimos todos los pasos para el lado derecho veremos que también es AB

AB - ¿Es AB?



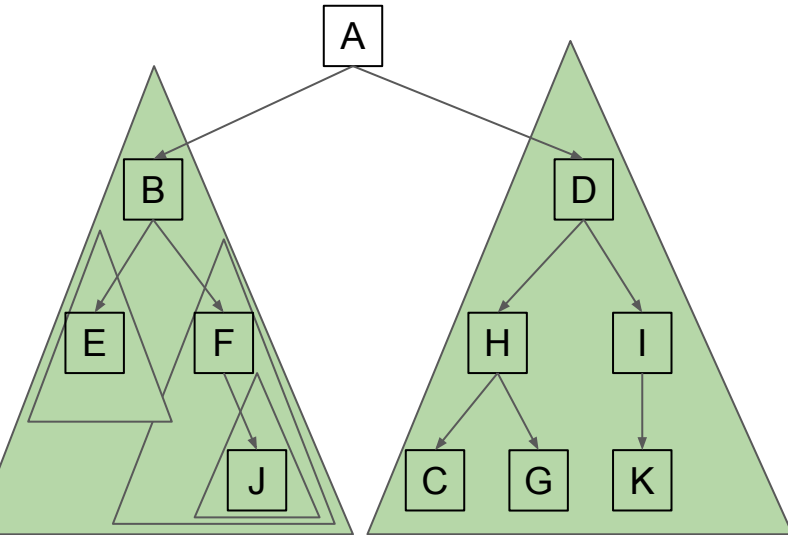
Subárbol
Izquierdo

Subárbol
Derecho

- Es de Grado 2.
- Sus subárboles son AB o son vacíos?

Si repetimos todos los pasos para el lado derecho veremos que también es AB

AB - ¿Es AB?



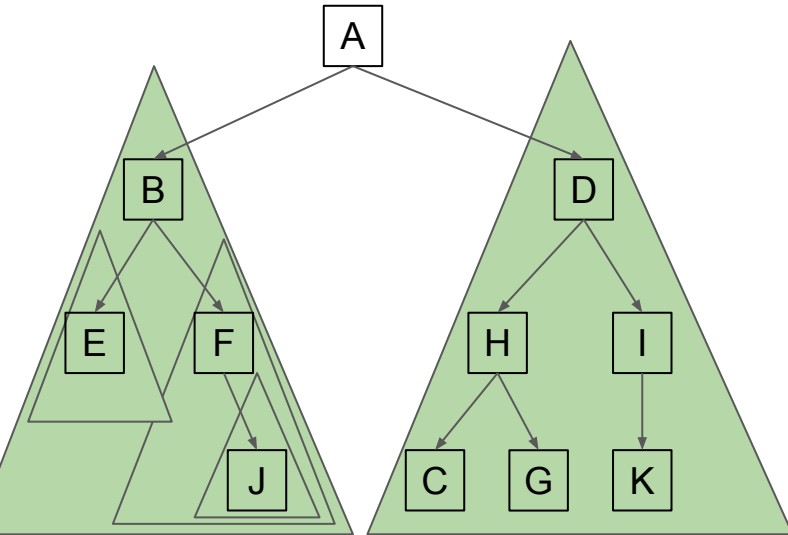
Subárbol
Izquierdo

Subárbol
Derecho

- Es de Grado 2.
- Sus subárboles son AB o son vacíos? “**SI**”

Si repetimos todos los pasos para el lado derecho veremos que también es AB

AB - ¿Es AB?



Subárbol
Izquierdo

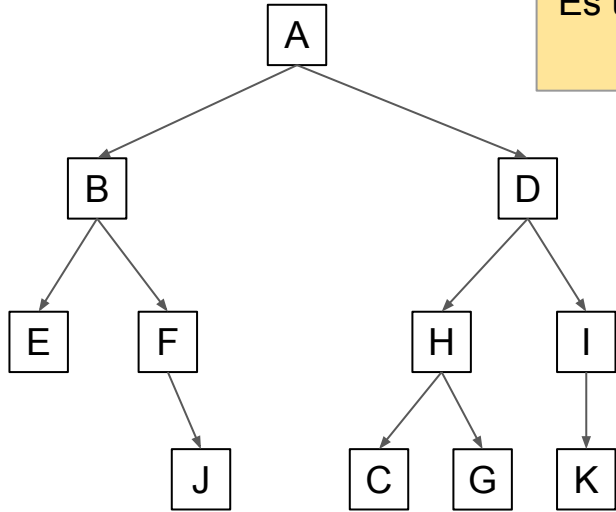
Subárbol
Derecho

- Es de Grado 2.
- Sus subárboles son AB o son vacíos? “**SI**”

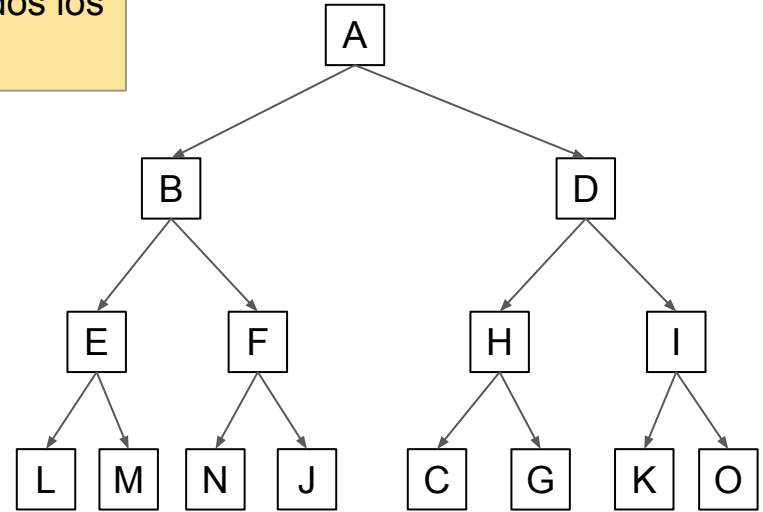
Pudimos comprobar que el
Árbol es un Árbol Binario.

AB - Completo

Es un árbol que tiene todos los niveles completos



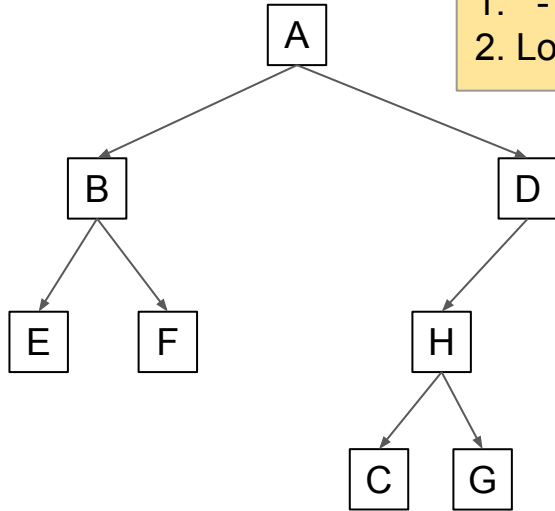
Árbol
incompleto



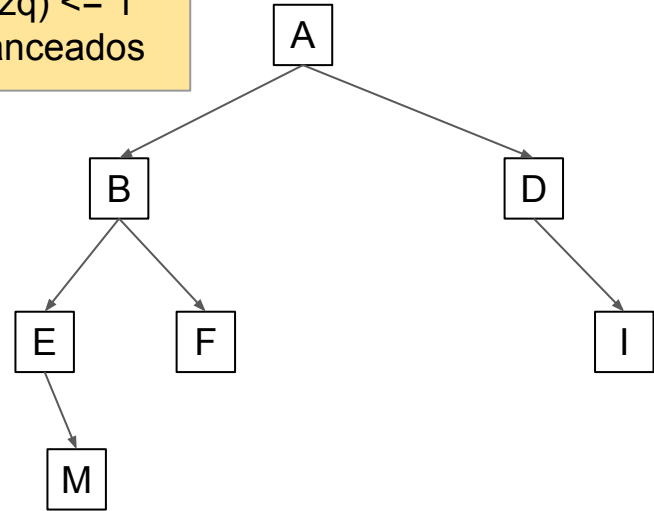
Árbol
completo

AB - Balanceado

1. $-1 \leq \text{altura}(\text{n.der}) - \text{altura}(\text{n.izq}) \leq 1$
2. Los dos subárboles están balanceados



Árbol
Desbalanceado



Árbol
Balanceado

AB - Conceptos

- Árbol Binario (Definición recursiva)
- Árbol Binario completo
- Árbol Binario balanceado (Definición recursiva)

¿Consultas?

Implementación de un AB

Como mencionamos anteriormente, un Árbol y sus subtipos son estructuras de datos. Esto significa que nos permitirán organizar nuestra datos de una forma en específica. Particularmente, en una estructura de árbol nos interesará poder realizar las acciones:

- void agregar(elemento)
- boolean pertenece (elemento)
- void quitar(elemento)
- int tamaño()
- int altura()
- boolean estaBalanceado()

Implementación de un AB - IREP

Para implementarlo se debe respetar el siguiente IREP:

- Se puede llegar a todos los nodos desde la raíz
- Cada nodo tiene un solo padre
- No tiene ciclos
- Cada nodo puede tener a lo sumo 2 hijos.

Implementación de un AB

```
public class AB<T> { // Árbol binario Genérico
    protected Nodo raiz;

    private class Nodo { // Clase interna
        T elem;
        Nodo izq, der;

        Nodo(T elem) {
            this.elem = elem;
        }
    }
    // ... MÉTODOS A IMPLEMENTAR
```

Implementación de un AB

```
// ... MÉTODOS A IMPLEMENTAR
public void agregar(T elem) {}           // 4to
public boolean pertenece(T elem) {}      // 2do
public void quitar(T elem) {}           // 5to
public int cantElementos() {}           // 1ro
public int altura() {}                  // 3ro
public boolean estaBalanceado() {}      // NORMAL
public boolean estaCompleto() {}        // HARDCORE
// suponiendo que es árbol de enteros.
public int minimo() {}                  // EASY
public int maximo() {}                  // EASY
```